SYSTEM ANALYSIS DOCUMENTATION (SAD)

What group name?

1. Introduction:

Currently, when team disruptions occur—such as sudden leave or employee overload—task redistribution depends heavily on human coordination, often resulting in delays, inefficiencies, and increased burnout among remaining team members.

Zenius addresses this challenge by operating within the domain of intelligent workflow orchestration and adaptive decision-making. It is designed as an AI-powered system that automates the detection of team disruptions and optimizes task distribution in real time.

The system leverages Z.AI's GLM as a central reasoning engine to ingest unstructured data from communication channels such as chats and emails. By analyzing signals related to absence or stress, Zenius autonomously makes decisions to reassign tasks based on skill matching and current workload, ensuring both operational continuity and team well-being.

1.1. Purpose:

The system analysis document highlights the technical scope and design-related decisions behind the Zenius development.

So, the key elements of the system that have been covered are as follows:

1. **Architecture**:
    a. It's a cloud-based system where services are separated into modular components.
    b. The system consists of three main layers:
        i. Input Layer (Sensors), Reasoning Engine, and Action Layer.
        ii. It is built using Python FastAPI (sensor services), Node.js backend, Firebase database, and Next.js frontend.
        iii. The AI reasoning is powered by Z.AI GLM-5, integrated as a service layer, and all components work together in a client-server architecture.
2. **Data Flows:**
    a. The system monitors external channels such as Gmail through sensor services.
    b. When a trigger is detected (e.g. stress message, leave notification, new task), it is sent to the backend API.
    c. The backend invokes the Reasoning Engine, which processes the data using AI and constraint-based logic.
    d. The results are stored in Firebase (Approvals collection) and then displayed on the frontend dashboard for managers and employees.
3. **Model Process:**
    a. The model shows the end-to-end workflow of the core features: detecting triggers, classifying intent using AI, calculating workload and burnout risk, and automatically generating task reassignment suggestions.

b. The workflow includes an approval mechanism where managers can review AI decisions before execution.

1.2. **Background**:

Company Z operates with multiple teams handling administrative and operational tasks across different departments. However, their workflow management is highly manual and dependent on managers to monitor employee availability, workload, and task assignments.

1. Modern teams rely heavily on manual task management and lack visibility into employee workload and well-being.
2. Managers often cannot detect early signs of stress or burnout, and task reassignment is inefficient and reactive.
3. Zenius addresses this gap by introducing AI-driven monitoring, reasoning, and automation to ensure balanced workload distribution.

1.3. Target Stakeholder

Stakeholders	**Roles**	**Expectations**
Project Manager	Monitor team workload, view AI-generated recommendations, and approve or adjust task reassignments	Real-time visibility of team health, accurate workload balancing, and control over final decisions
Employees	Receive task assignments, communicate updates, and signal workload or stress through system inputs	Fair task distribution, reduced burnout risk, and transparency in task changes

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	**Details**
System	Website

Architecture	Cloud-based Client-Server with Modular Layered Design

Zenius is structured as a client-server-based system where users interact via a web dashboard.

The system consists of three main components:

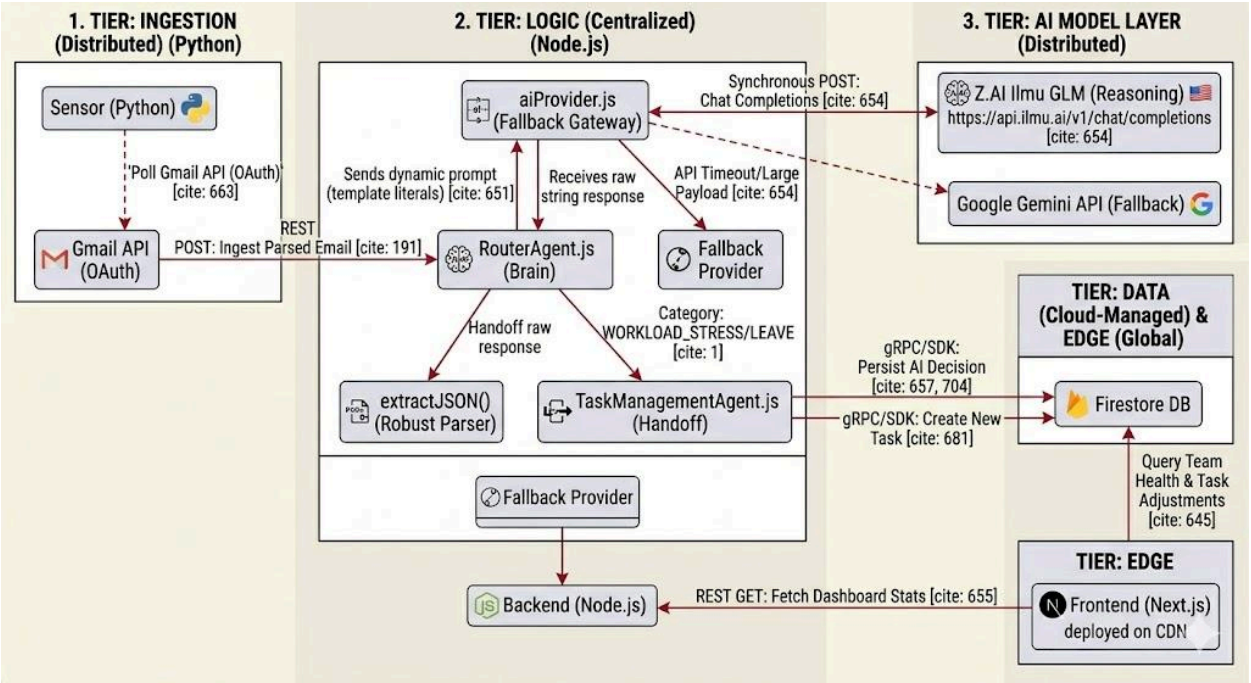
- Sensor Layer (Python FastAPI) for input ingestion
- Backend API Layer (Node.js) for orchestration
- Reasoning Engine (Python + GLM-5) for decision-making

All components communicate via APIs and store data in Firebase Firestore for real-time updates.

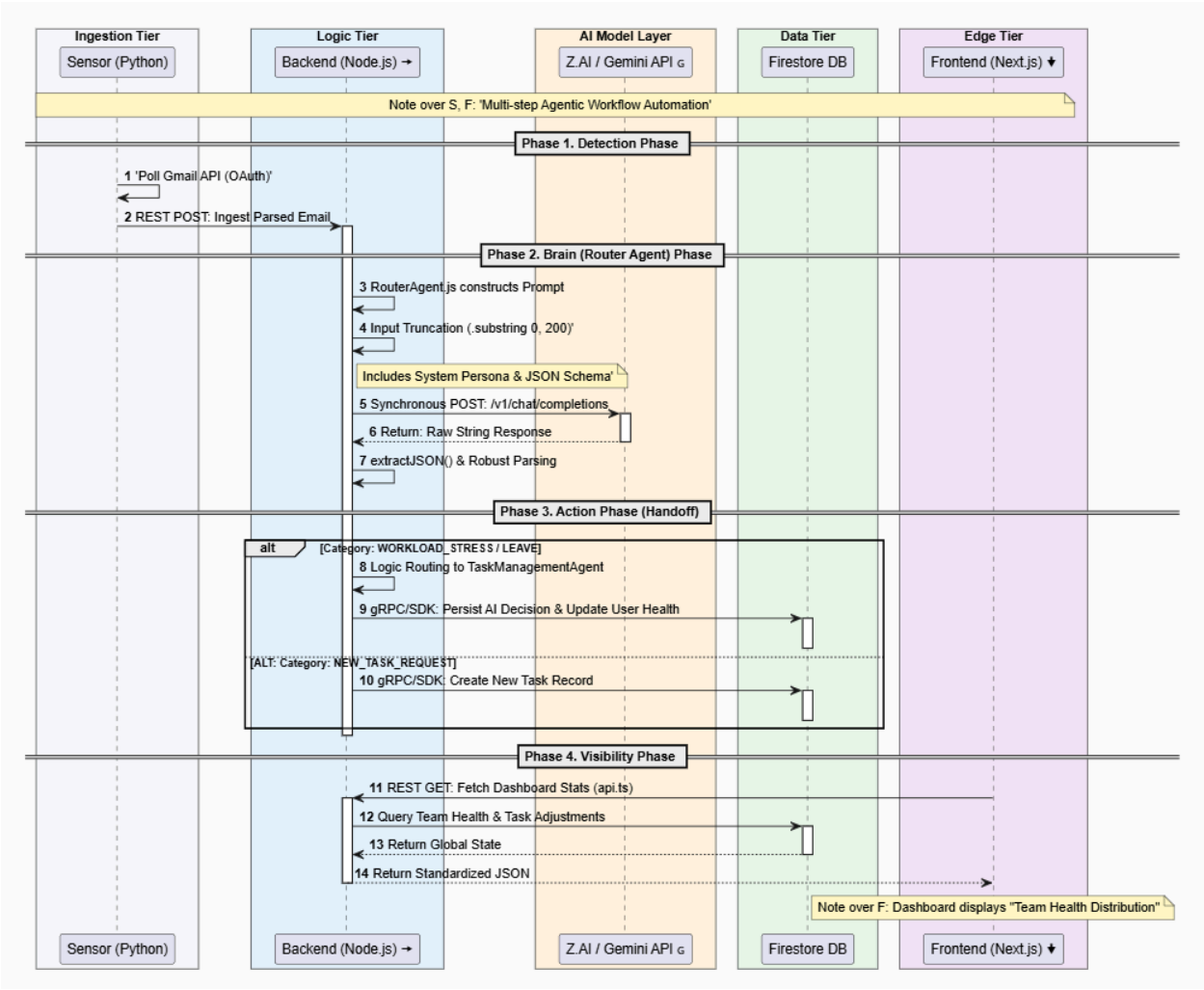
2.1.2 LLM as Service Layer

- Prompts are constructed using structured input data (email/chat content, user workload, sentiment scores)
- Context window includes:
  - User information (skills, load)
  - Task details
  - Trigger data (messages, events)

2.1.3 Dependency Diagram



2.2.2. Sequence Diagram



2.2. Technological Stack

2.2.1. Frontend - Next.js, web-based dashboard for managers and employees to visualize team workload, burnout risk, and AI-driven task adjustments. Tailwind CSS is used for UI styling, with Recharts for data visualization and Zustand for state management.

2.2.2. Backend – Node.js (Express), RESTful API layer that orchestrates system communication between frontend, AI services, and database. Handles routing, approval workflows, and task execution logic.

2.2.3. AI / Reasoning Engine - Z.AI GLM (Gemini fallback), core reasoning model used for classification, sentiment understanding, and decision-making. Integrated through a controlled prompt-based system (RouterAgent) with structured JSON output enforcement.

2.2.4. Sensor Layer - Python (FastAPI), lightweight ingestion service that collects unstructured data from Gmail API. Performs preprocessing, sentiment extraction, and structured data formatting before sending to backend.

2.2.5. Database - Firebase Firestore, NoSQL real-time database used to store users, tasks, input logs, and AI approval decisions. Supports real-time synchronization between backend and frontend dashboards.

2.2.6. Cloud/Deployment (will be implemented further)

2.2.6.1. Vercel – Frontend deployment for low-latency dashboard access

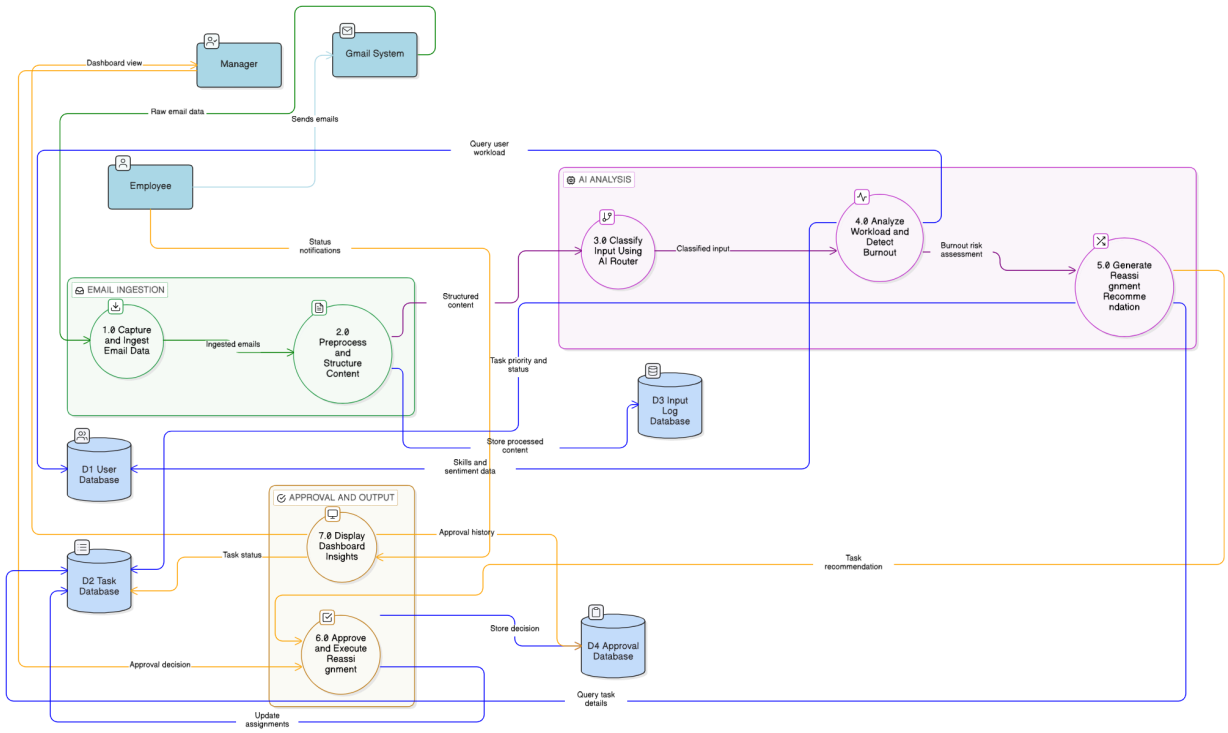
2.2.6.2. Google Cloud / Cloud Run – Hosting for sensor services and backend APIs

2.2.6.3. Firebase Cloud Infrastructure – Managed backend services for database and authentication

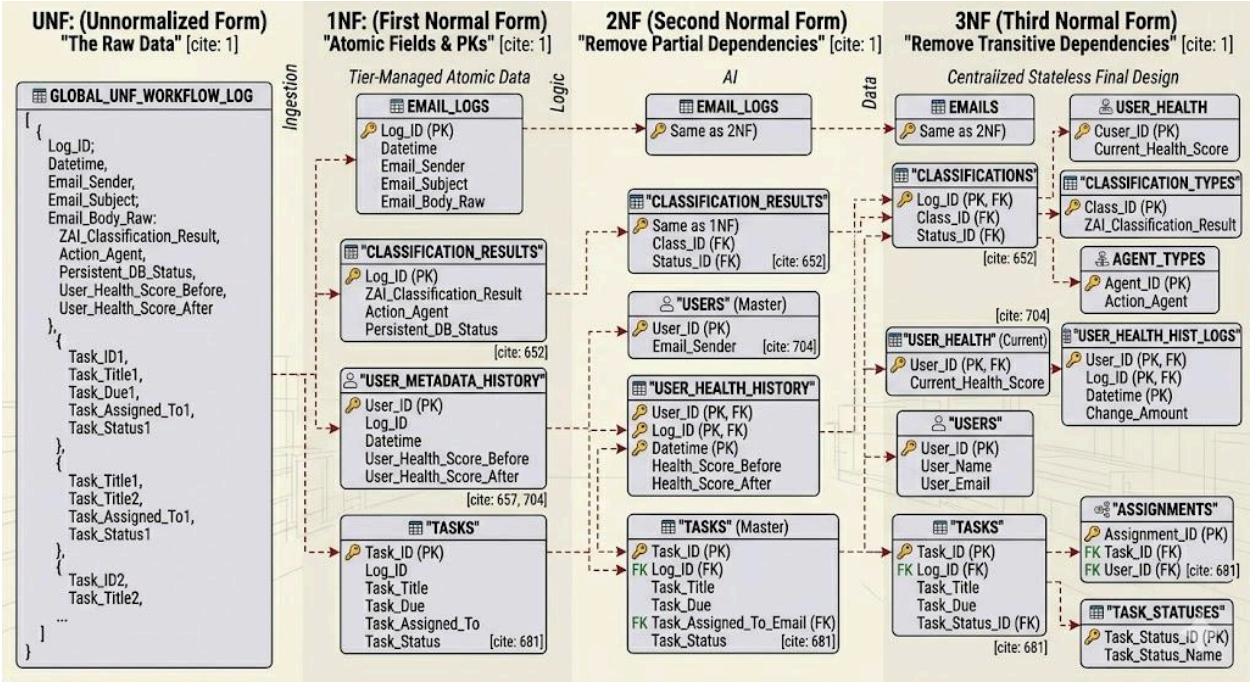
2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)



2.3.2. Normalized Database Schema



3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. 5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026.

#	Features	Description
1	Input Detection	Detects triggers from Gmail
2	AI Classification	Categorizes input using GLM-5
3	Burnout Detection	Calculates risk using load, sentiment, activity
4	Task Reassignment	Suggests optimal reassignment
5	Dashboard	Displays team health and task changes

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system.

Quality	Requirements	Implementation
Scalability	Handle multiple inputs and users in real-time	Firebase + modular services
Reliability	Accurate and stable AI decisions	Validation logic + fallback
Maintainability	Independent modules (sensor, backend, AI)	Modular architecture
Token Latency	AI response within acceptable time	Async calls + timeout control
Cost Efficiency	Control AI usage cost	Prompt optimization & truncation

3.3. Out of Scope / Future Enhancements

a. Integration with more platforms (Microsoft Teams, Zoom)

b. Predictive workload forecasting

c. Fully automated execution without approval

d. Cross-team optimization

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. **Grayscale Rollout & A/B Testing:** Our deployment strategy for the AI-powered workflow system utilizes a Grayscale Rollout to ensure stability. New features, such as the Router Agent's specialized categorization logic (e.g., detecting WORKLOAD_STRESS), are initially released to 5% of real users. We monitor system performance and AI response accuracy during this phase, gradually increasing the rollout to 100% if no anomalies are detected. Once the feature is fully stable, A/B testing is conducted to compare different Prompting Strategies (e.g., zero-shot vs. few-shot) to determine which variant performs better in terms of reasoning accuracy and token efficiency

4.1.2. **Emergency Rollback (ER) & Golden Release**: To manage recovery and traffic during emergencies, an Emergency Rollback (ER) is triggered if the system detects critical

failures, such as the GLM reasoning engine returning unusable or hallucinated responses. In these instances, the automated pipeline immediately deploys the Golden Version—a verified, stable baseline of the application—to maintain core service availability while the development team addresses the issue.

4.1.3. Priority Matrix:

4.1.3.1. The following matrix defines the trigger conditions and corresponding actions for our project to ensure high reliability and maintainability:

Priority Level	Trigger Condition	Mandatory Action
P1 - Critical	AI categorization error rate > 5% or API Timeout (30s) from Z.AI Ilmu GLM.	Immediate Emergency Rollback to the Golden Release and switch to the Fallback Provider (Google Gemini).
P2 - High	Average Token Latency exceeds 5 seconds for asynchronous API calls.	Trigger Input Truncation or chunking mechanisms to reduce payload size and decrease inference time.
P3 - Medium	Incorrect task reassignment suggestions identified by managers during the Approval Workflow.	Log the "False Positive" response in the Defect Log for iterative prompt tuning and context window refinement.
P4 - Low	Dashboard UI latency in the Frontend (Next.js) when fetching "Team Health" stats.	Monitor performance and implement Redis Caching for frequently queried task metadata.

4.2. **Monitoring**:

4.2.1. We implement a comprehensive monitoring plan to track the health of our Distributed Architecture:

4.2.1.1. System Health: Continuous tracking of the Python Sensor's polling frequency and the Node.js Backend's response time.

4.2.1.2. AI Output Pass Rate: Validating that at least 80% of documented prompts produce acceptable, structured JSON outputs.

4.2.1.3. Resource Monitoring: Using cloud-native tools to monitor Firestore DB read/write latencies and Cloud Run container health.

4.3. Assumptions: State key operational & environmental conditions that you assumed to be valid while development of system and deployment.

a. Network Stability: eUsers (Managers and Employees) have stable internet connections on their devices to access the Next.js Frontend and receive real-time updates from Firebase.
b. Email Provider Access: The system assumes continuous, authorized access to organizational Gmail accounts via the Python Sensor to monitor and ingest raw communication data.
c. Data Integrity: It is assumed that incoming emails contain sufficient context (e.g., clear intent or metadata) for the Router Agent to perform multi-step reasoning and categorization.
d. Hardware Readiness: The organization provides employees with compatible devices capable of running modern web browsers to view the Dashboard and manage "Task Adjustments".
e. User Cooperation: Employees are assumed to use the designated communication channels for work-related requests to allow the AI to accurately calculate burnout risk and load balancing

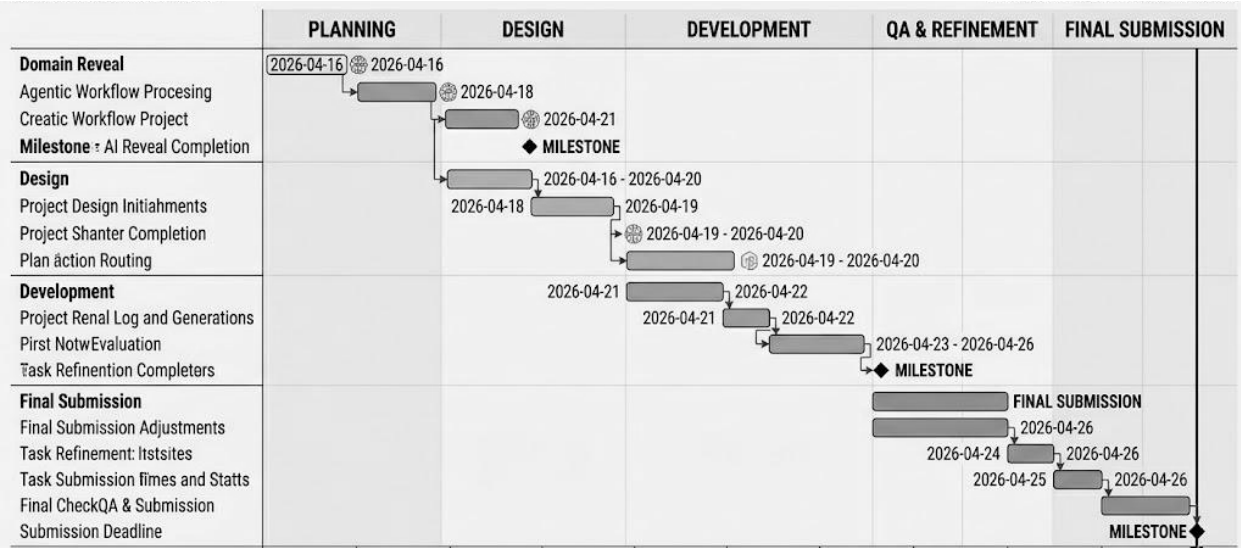
4.4. **External Dependencies:** Do list all the external tools that are applied to your system to function.

Tools	Purpose	Risks
Z Ai GLM API	It should be the core LLM reasoning engine - that will handle the prompt inference, multi-step reasoning and generating structured output	High- in case of GLM API rate limitation - core product will fail. So, implement a mitigation strategy [e.g. graceful degradation state from users]

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:



5.2. Team Members Role:

Name	Role	Key Responsibilities
Nicol Heng Si Yi	Database & Cloud Lead	Responsible for the integration of Firebase . Ensures every AI decision is persisted in Firestore and manages the Data Tier.
Phua Xin Wei	AI & Backend Lead	Responsible for AI and backend integration. Manages the Z.AI Ilmu GLM reasoning logic and the Node.js orchestration layer.
Tan Syn Yee	Ingestion & Sensor Lead	Responsible for the Python Sensor service. Handles Gmail API integration and the initial detection of unstructured inputs .
Chia Jing Yuen	UI/UX & Frontend Lead	Responsible for the Next.js dashboard. Focuses on Team Health visibility and user-facing "Task Adjustments".
Teo Jing Ying	Project Manager	Responsible for the Quality Assurance Testing Documentation (QATD) and tracking team progress. Ensures all materials meet UMHackathon 2026 standards.

5.3. Recommendations:

5.3.1. Scalability: Use a message queue (like RabbitMQ) so the Python sensor doesn't crash during busy email periods.

5.3.2. Performance: Use Redis Cache for team data to stop the system from constantly asking the database for the same info.

5.3.3. Reliability: Use a Circuit Breaker so if Z.AI fails, the system instantly switches to a Fallback Provider